

Callas.API Backend

Change Report — CallasFoundation (original) vs. Erykah Redesign

This report documents every functional difference found in the **Callas.API** ASP.NET Core backend between the original **CallasFoundation** project and the **callas-website-erykah-redesign** project. Cosmetic-only differences (Windows CRLF vs. Unix LF line endings, which affected nearly every file) were normalized out before comparison so that only real code changes are reported below.

Impact	Count	Examples
Major	3	Database engine swap, new password-change flow, admin-only write locks
Moderate	8	New content fields across News / Events / Team, new Health endpoint, CORS config
Minor / Cosmetic	6	Comment removal, expression-bodied methods, using-directive ordering

Contents

1. Database — SQL Server → SQLite
2. Authentication — Password-Change Endpoint
3. Authorization — Admin Locks on Write Endpoints
4. New Content Fields (News, Events, Team)
5. CORS Configuration
6. New Health-Check Endpoint
7. Project File & Package Changes
8. Minor / Cosmetic Cleanup
9. Unchanged Areas

1. Database — SQL Server → SQLite

Connection provider switched

MAJOR

The redesign replaces SQL Server (LocalDB) with SQLite as the data provider. This is a significant infrastructure change: it removes the dependency on a locally-installed SQL Server instance and lets the API run with a self-contained file database (`callas.db`), simplifying local setup and portability.

Files: `Program.cs`, `appsettings.json`, `Callas.API.csproj`

```
- options.UseSqlServer(cs);  
+ options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection"));
```

Connection string updated

MAJOR

The LocalDB connection string was replaced with a simple SQLite file path.

Files: `appsettings.json`

```
- "DefaultConnection":  
  "Server=(localdb)\MSSQLLocalDB;Database=CallasFoundation;Trusted_Connection=True;..."  
+ "DefaultConnection": "Data Source=callas.db"
```

Package references updated

MODERATE

Added the `Microsoft.EntityFrameworkCore.Sqlite` package to support the new provider. The `Microsoft.EntityFrameworkCore.Design` package block was removed. Note the old `Microsoft.EntityFrameworkCore.SqlServer` reference was left in place and is now unused — worth pruning.

Files: `Callas.API.csproj`

Startup now applies migrations automatically

MODERATE

In the development seeding block, the app now calls `db.Database.Migrate()` before checking/seeding the admin user, so the SQLite file is always brought up to date on startup. A stray debug line that printed the raw connection string to the console was also removed.

Files: `Program.cs`

```
- Console.WriteLine($"[{cs}]"); // removed  
+ db.Database.Migrate(); // added, runs before seeding
```

New EF Core migrations

MODERATE

The original project has a single `InitialCreate` migration. The redesign replaces it with `InitialSqlite` (matching the new provider) and adds three incremental migrations reflecting the new content fields described in Section 4: `AddNewsAuthorAndImage`, `AddEventImageUrl`, and `AddTeamBioAndImage`.

Files: `Migrations/*.cs`

2. Authentication — Password-Change Endpoint

New endpoint: PUT /api/auth/password

MAJOR

A new authenticated endpoint lets a logged-in user change their own password. It reads the caller's user ID from the JWT's NameIdentifier claim, so no user ID needs to be passed in the request body.

Files: Controllers/AuthController.cs

```
+ [Authorize] [HttpPut("password")] public async Task ChangePassword(ChangePasswordDto dto)
```

New service method: ChangePasswordAsync

MAJOR

Verifies the supplied current password against the stored hash using ASP.NET's PasswordHasher, and if valid, re-hashes and persists the new password. Returns false (mapped to a 400 response) if the current password is wrong.

Files: Interfaces/IAuthService.cs, Services/AuthService.cs

Repository support added

MODERATE

Two new methods were added to support the password-change flow: fetching a user by primary key, and a generic save that reports whether any rows were affected.

Files: Interfaces (Repositories)/IUserRepository.cs, Repositories/Auth/UserRepository.cs

```
+ Task GetByIdAsync(int id); Task SaveChangesAsync();
```

New DTO: ChangePasswordDto

MINOR

Carries CurrentPassword (required) and NewPassword (required, minimum 8 characters) for the new endpoint.

Files: DTOs/Auth/ChangePasswordDto.cs (new file)

Admin seed message improved

MINOR

The dev-only admin seeding logic now checks for an existing admin by username explicitly and logs to the console whether it created a new admin or skipped seeding because one already exists, instead of silently seeding only when the Users table is completely empty.

Files: Program.cs

```
- if (!db.Users.Any()) { ... }  
  
+ var existingAdmin = db.Users.FirstOrDefault(u => u.Username == "admin"); if  
(existingAdmin is null) { ... } else { Console.WriteLine("[seed] Admin user already  
exists..."); }
```

3. Authorization — Admin Locks on Write Endpoints

Events and Team endpoints now require Admin role

MAJOR

In the original project, the create/update/delete actions on Events and Team members had no `[Authorize]` attribute at all, meaning any anonymous caller could modify or delete that content. The redesign adds `[Authorize(Roles = "Admin")]` to the POST, PUT, and DELETE actions on both controllers, closing that gap.

Files: `Controllers/EventsController.cs`, `Controllers/TeamController.cs`

```
- [HttpPost] public async Task CreateEvent(...) // no auth guard
+ [Authorize(Roles = "Admin")] [HttpPost] public async Task CreateEvent(...)
```

Contact form attribute ordering

MINOR

The public contact-form POST action still allows anonymous submissions; the `[AllowAnonymous]` and `[HttpPost]` attributes were simply reordered. No behavioral change.

Files: `Controllers/ContactController.cs`

4. New Content Fields (News, Events, Team)

The redesign's frontend displays richer cards for news articles, events, and team members. To support that, matching fields were added end-to-end — model, Create/Update/Read DTOs, and service mapping logic — for each of the three entities below.

Entity	New Fields	Validation	Layers touched
News	Author, ImageUrl	Author: MaxLength(100) ImageUrl: [Url]	Model, 3 DTOs, Service
Event	ImageUrl	[Url]	Model, 3 DTOs, Service
Team Member	Bio, ImageUrl	Bio: MaxLength(1000) ImageUrl: [Url]	Model, 3 DTOs, Service

News: Author + ImageUrl

MODERATE

Added to the News model, all three DTOs (Create/Update/Read), and wired through `NewsService`'s create, update, and ToDto mapping.

Files: `Models/News.cs`, `DTOs/News/*.cs`, `Services/NewsService.cs`

Events: ImageUrl

MODERATE

Added to the Event model, all three DTOs, and wired through `EventService`'s create, update, and ToDto mapping.

Files: `Models/Event.cs`, `DTOs/Events/*.cs`, `Services/EventService.cs`

Team Members: Bio + ImageUrl

MODERATE

Added to the TeamMember model, all three DTOs, and wired through `TeamService`'s create, update, and ToDto mapping.

Files: `Models/TeamMember.cs`, `DTOs/Team/*.cs`, `Services/TeamService.cs`

5. CORS Configuration

Allowed origins moved to configuration

MODERATE

The original project hardcoded a single allowed CORS origin (the Vite dev server on port 5173) directly in `Program.cs`. The redesign reads a list of allowed origins from `Cors:AllowedOrigins` in configuration, falling back to ports 5173 and 3000 if the setting is absent. This makes it possible to add environments (e.g. staging, a second dev server) without touching code.

Files: `Program.cs`, `appsettings.Development.json`

```
- policy.WithOrigins("http://localhost:5173")

+ var origins = builder.Configuration.GetSection("Cors:AllowedOrigins").Get() ?? new[] {
  "http://localhost:5173", "http://localhost:3000" }; policy.WithOrigins(origins)
```

Development origins expanded

MINOR

`appsettings.Development.json` now explicitly lists three allowed origins: 5173, 3000, and 8080.

Files: `appsettings.Development.json`

6. New Health-Check Endpoint

GET /api/health

MODERATE

A new, unauthenticated controller returns a simple JSON payload confirming the API is up. Useful for uptime monitors, container orchestration health probes, or load balancer checks — none of which existed in the original project.

Files: `Controllers/HealthController.cs` (new file)

```
+ [HttpGet] public IActionResult Get() => Ok(new { ok = true });
```

7. Project File & Package Changes

.gitignore and .http file tweaks

MINOR

Small formatting/content differences in the git-ignore rules and the saved HTTP request-collection file; no impact on runtime behavior.

Files: `.gitignore`, `Callas.API.http`

Callas.API.csproj.user removed

MINOR

The user-specific IDE settings file present in the original project is absent from the redesign (as expected — this file is machine/IDE-local and typically git-ignored).

Files: `Callas.API.csproj.user`

Empty Middleware/ and wwwroot/ folders removed

MINOR

Both directories existed in the original project but contained no files, so their removal has no functional effect.

Files: `Middleware/`, `wwwroot/`

8. Minor / Cosmetic Cleanup

Beyond the functional changes above, the redesign applies light, repo-wide style cleanup. None of this affects behavior:

- Removed most blank lines separating class members and explanatory code comments (e.g. the long comment block above service registrations in Program.cs, the XML-doc comments on AuthController's Login/Logout).
- Reordered some using directives for consistency (Authorization before Mvc).
- Converted a few simple controller actions to expression-bodied members, e.g. `ContactController.GetAllMessages()`.
- Renamed a couple of local variables for brevity (updated/deleted → ok) in ContactController.
- Collapsed multi-line JSON formatting in appsettings.Development.json onto fewer lines.

9. Unchanged Areas

The following controllers, services, and repositories are functionally identical between the two projects (only line-ending differences were present, which were normalized out before comparison): **Donations, Gallery, Impact, Partners, Programmes, Resources, and Volunteers** — covering their controllers, services, interfaces, repositories, and DTOs. Properties/launchSettings.json is also unchanged.

Generated by comparing CallasFoundation.zip (original) against callas-website-erykah-redesign.zip, scoped to the Callas.API project only.